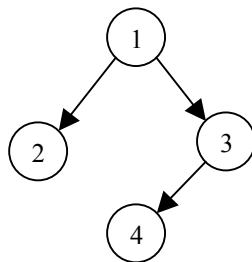


Задача 7. (14 т.) Нека е дадено следното представяне на двоично дърво с произволни стойности по върховете:

- празният списък $()$ е празно дърво
- ако t_1 и t_2 са две двоични дървета, то списъкът с три елемента $(x \ t_1 \ t_2)$ е двоично дърво със стойност на корена x , ляво поддърво t_1 и дясно поддърво t_2 .

Да се дефинира функция (*leaves* t), намираща списък от стойностите по листата на дървото t , представено по писания начин.

Пример: Следното двоично дърво



се представя чрез списък $t = (1 \ \underline{(2 \ () \ ())} \ \underline{(3 \ (4 \ () \ ()) \ ())})$. За него (*leaves* t) = (2 4).

Задача 5. (15 т.) Списъкът $l_1 = (u_0..u_k)$ наричаме *подписък* на списъка $l_2 = (v_0..v_n)$, ако $k \leq n$ и съществува такова i , че $0 \leq i \leq n - k$ и $u_0 = v_i, u_1 = v_{i+1}, \dots, u_k = v_{i+k}$. Пример: списъкът (1 2) е подписък на списъка (0 1 2 3), но не е подписък на списъка (1 0 2).

Да се дефинира функция **(count-sub l1 l2)** на езика Scheme, която намира колко пъти списъкът от числа $l1$ се среща като подписък на списъка от числа $l2$.

Пример: (count-sub '(1 1) '(1 1 1 2 1 1)) -> 3

Задача 7. (12 точки) Компресирано представяне на даден списък от стойности наричаме такъв списък от точкови двойки, за който неколкото последователно срещане на един елемент е заменено от точкова двойка, първият елемент на която показва повторената стойност, а вторият – броя на повторенията. Например, некомпресираният списък **(2 2 2 Stan 7 7 2)** след компресия приема вида **((2 . 3) (Stan . 1) (7 . 2) (2 . 1))**. Да се реализират на езика Scheme двойка функции, които преобразуват некомпресиран списък в компресиран и обратно.

Задача 8. (15 точки) Компресирано представяне на безкраен поток от числа наричаме такъв друг поток, за който неколкото последователно срещане на числото 0 е сменено от точкова двойка с първи елемент 0 и втори – броят на повторенията на 0. Например, ако началото на един безкраен поток е (0 0 0 1 0 0 7 3 . . . , компресираното му представяне е безкраен поток с начало ((0 . 3) 1 (0 . 2) 7 3 Да се реализират на езика Scheme двойка функции, които преобразуват некомпресиран безкраен поток в компресиран и обратно.

Задача 6. (12 точки) Нека е даден списък от естествени числа L , който съдържа цифри (естествени числа в интервала $[0, 9]$). Напишете функция (`find-max L`), която намира най-голямото число, което може да се образува от цифрите в L . Ако L е празен, функцията `find-max` трябва да връща нула.

Пример:

`(find-max '())` $\rightarrow$ 0

`(find-max '(0 0 0))` $\rightarrow$ 0

`(find-max '(1 1 9 8 9 3 4 6 7 0 0))` $\rightarrow$ 99876431100

Задача 6. (12 точки) Да се дефинира на езика Scheme функция (`generate-bin n`), която генерира поток от всички естествени числа в интервала $[n, +\infty)$, представени чрез своя двоичен запис. Двоичният запис за едно число се представя като списък от нули и единици. Например:

- $1 \rightarrow (1)$
- $2 \rightarrow (1\ 0)$
- $10 \rightarrow (1\ 0\ 1\ 0)$

Примерни изпълнения:

`(generate-bin 0)` $\rightarrow$ Потокът е $(0)\ (1)\ (1\ 0)\ (1\ 1)\ (1\ 0\ 0)\ (1\ 0\ 1)\ \dots$

`(generate-bin 10)` $\rightarrow$ Потокът е $(1\ 0\ 1\ 0)\ (1\ 0\ 1\ 1)\ (1\ 1\ 0\ 0)\ (1\ 1\ 0\ 1)\ \dots$

Задача 7. (10 т.) *Задачата да се реши на езика Scheme или Haskell. В началото на вашето решение посочете кой език сте избрали.*

А) Напишете функция `totalMin`, която за списък от едноместни числови функции връща тази функция f от списъка, за която $f(0)$ е минимално.

Б) Напишете функция `chainMinCompositions`, която получава като аргумент едноместна числова функция f и генерира безкрайния поток (за Хаскел – безкрайния списък) $F_0, F_1, F_2, \dots$, където:

$$F_0 = id$$

$$F_1 = f$$

$$F_i = F_{i-1} \circ F_{i-2}, \text{ ако } i > 1 \text{ и } F_{i-1}(j) \neq F_{i-2}(j), \text{ за някое цяло число } j \in [0, i]$$

$$F_i = totalMin \{F_0, F_1, \dots, F_{i-1}\}, \text{ ако } i > 1 \text{ и } F_{i-1}(j) = F_{i-2}(j), \text{ за всяко цяло число } j \in [0, i]$$

Забележка: с id е означена функцията „идентитет“, като $id(x) = x$ за произволно x , а с $f \circ g$ е означена композицията на функциите на f и g , като $(f \circ g)(x) = f(g(x))$.

Задача 7. (10 т.) *Задачата да се реши на езика Scheme или Haskell. В началото на вашето решение посочете кой език сте избрали.*

Нека е даден списък L , който може да съдържа елементи от произволен тип. Напишете функция `permutations`, която получава такъв списък и връща списък с всички пермутации (възможни пренареждания) на неговите елементи. Резултатът да се върне като списък от списъци, в който всеки подсписък представя една пермутация на елементите на L .

Пример (Scheme):

`(permutations '(1 2 3)) → ( (1 2 3) (1 3 2) (2 1 3) (2 3 1) (3 1 2) (3 2 1) )`

Пример (Haskell):

`permutations [1,2,3] → [ [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1] ]`

Задача 3 (10 точки). Нека са дадени следните изрази на езиците Haskell и Scheme. Моля, посочете каква е оценката на изразите на един от двата езика по ваш избор (попълнете едно от правоъгълните карета по-долу).

```
map (head [(\couple->fst couple + snd couple)])  
    (foldr1 (++) [[(1,2)],[(3,4)]])
```

Оценка: _____

```
[zip [x] [x] | x <- [1..5]]
```

Оценка: _____

```
map (\(x:y:z)->x:z) [[1,2,3],[2,3,1],[3,1,2]]
```

Оценка: _____

```
(map  
  (car (list (lambda (couple) (+ (car couple) (cdr couple)))))  
  (apply append '( ( (1 . 2) ) ( (3 . 4) ) ) ) )
```

Оценка: _____

```
(map (lambda (x)  
      (cons x (list x)))  
  '(1 2 3 4 5))
```

Оценка: _____

```
(map (lambda (pred) (filter pred '(1 2 3 4 5)))  
  (list even? odd?))
```

Оценка: _____

Задача 3 (10 точки). Дадени са следните дефиниции на програмните езици Haskell и Scheme, от програмния код на които липсват части. Попълнете полетата, обозначени с _____ с необходимия програмен код така, че да се получат посочените желани оценки. Изберете само един от двата езика за решението на задачата и напишете името му в даденото за целта поле. Точки за задачата се дават само за изборния от вас език.

Избран език:

	Език Haskell:
1	merge x [] = _____ merge _____ = _____ merge (x:xs) (y:ys) = if x < y then _____ else _____ израз: merge [1,3,5,7] [2,2,6,10] желана оценка: [1,2,2,3,5,6,7,10]
2	израз: (_____ -> [y y<-_____, even _____]) [1,2,3,4] желана оценка: [2,4]
	Език Scheme:
1	(define (merge l1 l2) (cond ((null? l1) _____) (_____ ((< (car l1) (car l2)) _____) (else _____)))) израз: (merge '(1 3 5 7) '(2 2 6 10)) желана оценка: (1 2 2 3 5 6 7 10)
2	израз: ((lambda (_____) (filter _____)) '(1 2 3 4)) желана оценка: (2,4)

Задача 3. Нека са дадени следните изрази на езиците Haskell и Scheme. Да се посочи каква е оценката на изразите. Изберете само един от двата езика за решението на задачата и напишете името му в даденото за целта поле. Точки за задачата се дават само за избрани от Вас език.

Избран език:

Haskell:

1. `[ x : [x] | x <- [ [1,2], [3,4] ] ]`

Оценка:

2. `[ map (f 5) [1,2,3] | f <- [ (+), (-), (*) ] ]`

Оценка:

3. `"a" : [ ['b', 'c'], "d" ]`

Оценка:

Scheme:

1. `(map (lambda (x) (append (list x) x)) '( (1 2) (3 4) ) )`

Оценка:

2. `(map (lambda (f) (map (lambda (x) (f 5 x)) '(1 2 3) ) ) (list + - *))`

Оценка:

3. `(apply list (list (quote +) (quote 5) 8) )`

Оценка:

Задача 3.

1. Дадени да следните дефиниции на функция, съответно на програмните езици Haskell и Scheme, от програмния код на които липсват части. Да се попълнят полетата, обозначени с _____, с необходимия програмен код така, че да се получат посочените желани оценки.
2. Дадени са следните изрази, съответно на програмните езици Haskell и Scheme. Да се посочи каква е оценката на израза.

Изберете само един от двата езика за решението на задачата и напишете името му в даденото за целта поле. Точки за задачата се дават само за изборания от Вас език.

Избран език:

Haskell:	
1.	filterByChar c ls = filter _____ ls израз: filterByChar 'o' ["cat", "cow", "dog"] желана оценка: ["cow", "dog"]
2.	let (x:y):z = ["Curry"] in (x,y,z) Оценка:
Scheme:	
1.	(define (filterByElement x m) (filter (lambda (__) _____) m)) израз: (filterByElement 2 '((1 2 3) (2 3 4) (3 4 5))) желана оценка: ((1 2 3) (2 3 4))
2.	(apply + (map (lambda (l) (apply max l)) '((5 -2) (1 9) (6 -8)))) Оценка:

Задача 3. Задачата да се реши на един от езиците Scheme или Haskell. По-долу оградете името на езика, който сте избрали за вашето решение.

Дадени са непразен списък от едноместни числови функции **f1** и непразен списък от числа **x1**. Казваме, че числото **x** е “неподвижна точка” на функцията **f**, ако **f(x) = x**. Да се попълнят по подходящ начин празните полета по-долу така, че за всички функции от **f1**, които имат неподвижна точка сред числата в **x1**, функцията **sumMinFix** да намира сумата на най-малките им такива неподвижни точки. Ако никоя функция от **f1** няма неподвижна точка сред числата в **x1**, функцията **sumMinFix** да връща числото 0. Помощната функция **addDefault** служи да осигури, че ако подаденият ѝ списък е празен, то в него се добавя една стойност по подразбиране. **Упътване:** можете да използвате наготово функциите **apply**, **filter**, **foldr**, **map**, **min**, **minimum**, както и всички стандартни функции в R⁵RS за Scheme и Prelude за Haskell.

Scheme

```
(define (addDefault val l)
  (if (null? l) (list val) l))

(define (sumMinFix f1 x1)
  (_____
    (_____
      (lambda (f)
        (apply _____
          (addDefault _____
            (_____
              (lambda (x) _____ ) x1)))) f1))))
```

Пример:

```
(sumMinFix (list (lambda (x) (/ 1 x)) exp (lambda (x) (- (* 2 x) 3))))
'(-2 -1 1 3) → 2 (= -1 + 3)
```

Haskell

```
addDefault val [] = [val]
addDefault val l  = l
```

```
sumMinFix f1 x1 =
```

```
  _____
  ( _____
    (\f -> _____
      (addDefault _____
        [ _____ | x <- x1, _____ ])) f1)
```

Пример:

```
sumMinFix [ (1/), exp, \x -> 2*x - 3 ] [-2, -1, 1, 3] → 2 (= -1 + 3)
```

Задача 3. Задачата да се реши на един от езиците *Scheme* или *Haskell*. По-долу оградете името на езика, който сте избрали за вашето решение.

Даден е списък от списъци от числа **l1** и числова функция **f**. Числото **x** наричаме “корен” на **f**, ако **f(x) = 0**. Да се попълнят по подходящ начин празните полета по-долу, така че функцията **sumMaxRoots** да намира сумата на корените на **f** в този списък от **l1**, в който **f** има най-много корени. Ако има няколко такива списъка, функцията да връща сумата на корените в първия по ред списък. Ако функцията няма корен сред числата в списъците на **l1**, функцията да връща **0**.

Упътване: можете да използвате наготово функциите **apply**, **filter**, **foldr**, **length**, **map**, **max**, **maximum**, както и всички стандартни функции в R⁵RS за *Scheme* и *Prelude* за *Haskell*.

Scheme

```
(define (selectList l1 l2)
  (if _____ l1 l2))

(define (sumMaxRoots f l1)
  (_____
    (_____ selectList _____
      (_____ (lambda (l)
        (_____ (lambda (x) _____) l)) l1))))
```

Пример:

(sumMaxRoots (lambda (x) (- (* x x x) x)) '((1 2 3) (-1 0 5) (1 4 -1))) → -1

Haskell

```
selectList l1 l2 = if _____ then l1 else l2

sumMaxRoots f l1 =
  _____
  (_____ selectList _____
    (_____ (\l -> [ _____ | x <- l, _____ ])) l1))
```

Пример:

sumMaxRoots (\x -> x³ - x) [[1, 2, 3], [-1, 0, 5], [1, 4, -1]] → -1

Задача 3. Задачата да се реши на един от езиките *Scheme* или *Haskell*. По-долу оградете името на езика, който сте избрали за вашето решение.

Големият онлайн магазин *Siberia* търси начин да увеличи продажбите като препоръчва на клиентите си подходящи продукти. За целта изследователският екип на *Siberia* експериментира с различни реализации на функция **bestFit**, която приема като параметър код на продукт **a** и връща код на друг продукт **b**, който клиентите на магазина най-вероятно биха си купили заедно с **a**. Задачата пред разработчиците на *Siberia* е да реализират функция **recommended**, която получава като параметри потребителска кошница **basket** (списък от целочислени кодове на продукти), функция **bestFit** и списък от продуктите на магазина **products** (списък от наредени двойки от уникален код на продукт и цена — неотрицателно число).

Да се попълнят по подходящ начин празните полета по-долу така, че функцията **recommended** да връща списък от кодовете на всички възможни препоръчани продукти. Допуска се в резултата някои кодове да се срещат повече от веднъж. Препоръчан продукт е такъв, който:

- все още не е в **basket**, но се получава като резултат от прилагането на функцията **bestFit** над някой от продуктите, които вече са в **basket**;
- има цена, която не надвишава общата цена на потребителската кошница, дефинирана като сумата от цените на продуктите в **basket**.

Помощните дефиниции **findPrice** и **basketCost** намират съответно цената на даден продукт **product** в списъка **products** и цената на потребителската кошница. Да се приеме, че **basket** съдържа само кодове на продукти в **products** и **bestFit** също връща само такива кодове.

Упътване: могат да се използват наготово функциите **apply**, **assoc**, **elem**, **filter**, **foldr**, **lookup**, **map**, **member**, **sum** и стандартните функции в R⁵RS за *Scheme* и в Prelude за *Haskell*.

Scheme

```
(define (recommended basket bestFit products)
```

```
  (define (findPrice product)
```

```
    _____ products _____
```

```
  (define basketCost
```

```
    _____ basket _____
```

```
  (_____
```

```
    (lambda (product)
```

```
      _____)
```

```
    (_____ basket)))
```

Haskell

```
recommended basket bestFit products =
```

```
  _____
```

```
  (\product ->
```

```
    _____)
```

```
  (_____ basket)
```

```
  where findPrice product =
```

```
    _____ products _____
```

```
  basketCost =
```

```
    _____ basket _____
```

Задача 3. Задачата да се реши на един от езиците Scheme или Haskell. По-долу оградете името на езика, който сте избрали за решението си.

“Етикет” наричаме наредена двойка от низове — име и стойност, а “анотирана данна” наричаме наредена двойка от данна и списък от етикети за нея. Разглеждаме база данни, представена като списък от анотирани низове. “Анотатор” наричаме функция, която приема данна (низ) и връща списък от етикети за нея. Да се попълнят по подходящ начин празните полета по-долу, така че при подаване на база данни db и списък от анотатори annotators, функцията annotate да връща актуализирана база данни, в която за всяка данна в db са добавени етикетите, върнати за нея от анотаторите в annotators. Ако даден етикет вече съществува за дадена данна, той да не се добавя повторно. Да се реализира помощната функция addIfNew, така че да добавя елемента в x в началото на списъка l само ако x не се среща вече в l.

Упътване: могат да се използват наготово функциите append, apply, concat, concatMap, elem, filter, foldr, map, member, sum и стандартните функции в R5RS за Scheme и в Prelude за Haskell.

Scheme

```
(define (addIfNew x l) _____)
(define (annotate db annotators)
  (_____
    (lambda (item-labels-pair)
      (let ((item (car item-labels-pair)) (labels (cdr item-labels-pair)))
        (cons item (_____ addIfNew labels
          (_____
            (lambda (annotator) _____) annotators)))))) db))
```

Пример:

```
(define db (list (cons "scheme" (list (cons "typing" "dynamic") (cons "evaluation" "strict"))
  (cons "haskell" (list (cons "typing" "static")) (cons "c++" (list))))))
(define (evaluation lang)
  (case lang (("scheme") (list (cons "evaluation" "strict") (cons "macros" "true")))
    (("haskell") (list (cons "evaluation" "lazy"))) ("c++") (evaluation "scheme"))))
(define (purity lang) (if (eqv? lang "haskell") (list (cons "pure" "true")) (list)))
(annotate db (list evaluation purity)) →
(("scheme" ("macros" . "true") ("typing" . "dynamic") ("evaluation" . "strict"))
 ("haskell" ("evaluation" . "lazy") ("pure" . "true") ("typing" . "static"))
 ("c++" ("evaluation" . "strict") ("macros" . "true")))
```

Haskell

```
addIfNew x l = _____
annotate db annotators =
  _____
  (\(item, labels) ->
    (item, _____ addIfNew labels
      (_____
        (\annotator -> _____) annotators))) db
```

Пример:

```
db = [("scheme", [("typing", "dynamic"), ("evaluation", "strict")]),
      ("haskell", [("typing", "static"), ("c++", [])])
evaluation "scheme" = [("evaluation", "strict"), ("macros", "true")]
evaluation "haskell" = [("evaluation", "lazy")]
evaluation "c++" = evaluation "scheme"
purity lang = if lang == "haskell" then [("pure", "true")] else []
annotate db [evaluation, purity] →
[ ("scheme", [ ("macros", "true"), ("typing", "dynamic"), ("evaluation", "strict") ] ),
  ("haskell", [ ("evaluation", "lazy"), ("pure", "true"), ("typing", "static") ] ),
  ("c++", [ ("evaluation", "strict"), ("macros", "true") ] ) ]
```

Задача 3. Задачата да се реши на един от езиците *Scheme* или *Haskell*. По-долу оградете името на езика, който сте избрали за решението си.

При планински преход GPS устройствата могат да записват “следа” на изминатия път във вид на поредица от поне две точки. Всяка точка се задава с наредена двойка от цели числа: изминато разстояние от началото на прехода и надморска височина. Точките в следата са подредени в строго нарастващ ред по изминатото разстояние. “Наклон” на участъка между две точки наричаме абсолютната стойност на отношението на разликата във височините им и дължината на изминатия път между тях.

Да се попълнят по подходящ начин празните полета в програмата по-долу, така че при извикване на функцията `easiestTrackUnder` с максимална дължина `maxLen` и списък от следи `tracks` да се връща следата, чиито максимален наклон е възможно най-малък от всички следи от `tracks`, които са по-къси от `maxLen`. Ако има повече от една следа, отговаряща на условието, е без значение коя от тях ще бъде върната като резултат. Ако няма нито една следа, отговаряща на условието, поведението на функцията е недефинирано (може да бъде произволно).

Да се реализират помощните функции `maxSlope`, която намира максималния наклон на участък в пътека и `argMin`, която намира елемента в непразния списък `l`, над който функцията `f` дава минимална стойност.

Упътване: могат да се използват наготово функциите `abs`, `apply`, `filter`, `foldr`, `foldr1`, `last`, `map`, `min`, `minimum`, `max`, `maximum`, `reverse`, `zip`, както и всички стандартни функции в *R5RS* за *Scheme* и в *Prelude* за *Haskell*.

Scheme

```
(define (argMin f l) (_____ l))

(define (maxSlope track)
  (_____ (map (lambda (_____) _____)
                (reverse (cdr track)) (cdr (reverse track)))))

(define (easiestTrackUnder maxLen tracks)
  (argMin maxSlope (_____ (lambda (track) _____) tracks)))
```

Пример:

```
(define tracks '(((0 . 900) (100 . 910) (200 . 925) (300 . 905) (600 . 950))
                  ((0 . 1300) (100 . 1305) (500 . 1340) (800 . 1360) (1000 . 1320))
                  ((0 . 800) (200 . 830) (300 . 845) (600 . 880) (800 . 830))))

(maxSlope (car tracks)) → 0.2
(easiestTrackUnder 800 tracks) → ((0 . 900) (100 . 910) (200 . 925) (300 . 905) (600 . 950))
```

Haskell

```
argMin f l = _____ l

maxSlope track = _____
                  (map (\_____ -> _____)
                      (zip track (tail track)))

easiestTrackUnder maxLen tracks =
  argMin maxSlope (_____ (_____ tracks))
```

Пример:

```
tracks = [(0, 900), (100, 910), (200, 925), (300, 905), (600, 950)],
          [(0, 1300), (100, 1305), (500, 1340), (800, 1360), (1000, 1320)],
          [(0, 800), (200, 830), (300, 845), (600, 880), (800, 830)]

maxSlope (head tracks) → 0.2
easiestTrackUnder 800 tracks → [(0, 900), (100, 910), (200, 925), (300, 905), (600, 950)]
```

Задача 3. Задачата да се реши на един от езиците *Scheme* или *Haskell*.

Музикално произведение се описва с наредена тройка от име на автор, заглавие на произведението и дължина в секунди. Даден е непразен списък от музикални произведения *pl*, в който **те са подредени в нарастващ ред по дължина**. Препоръчваща функция наричаме функция, която приема наредена тройка, описваща текущо възпроизвеждащо се музикално произведение и връща заглавие на произведение, което се препоръчва да бъде следващото възпроизведено. Компанията *lify* е патентовала алгоритъм за препоръчваща функция, която за текущо възпроизвеждащо се произведение *p* с първи компонент (автор) *a* работи по следния начин за списък *pl*:

1. Препоръчва следващото по дължина произведение на *a* в *pl*, ако има такова.
2. В противен случай, препоръчва произведение с максимална дължина в *pl* из произведенията на всички автори, за които средната дължина на произведенията в *pl* е по-малка от средната дължина на произведенията на *a* в *pl*, ако има такива.
3. В противен случай, препоръчва следващото по дължина произведение в *pl* след *p*, а ако такова няма, препоръчва отново *p*.

Да се попълнят празните полета по-долу така, че функцията *recommender pl* да връща препоръчваща функция, реализираща алгоритъма на *lify*, описан по-горе, върху списък *pl*. Функцията *avgDuration* да връща средната дължина на произведенията на *author* в *pl*. Списъкът *option1* да съдържа произведенията на *a* с по-голяма дължина от *p*. Списъкът *option2* да съдържа произведенията на всички автори със средна дължина на произведенията по-малка от средната дължина на произведенията на *a*.

Упътване: могат да се използват наготово функциите *append*, *apply*, *filter*, *foldr*, *fromIntegral*, *length*, *list*, *last*, *map*, *reverse*, *sum*, както и всички функции в *R5RS* за *Scheme* и в *Prelude* за *Haskell*.

Scheme

```
(define (recommender pl)

  (define (avgDuration artist) _____)
  (define option1 _____)
  (define option2 _____)
  (if (not (null? option1)) _____ option1
      (if (not (null? option2)) _____ option2
          _____)))
```

Пример:

```
(define rf (recommender '(("Mozart" "The Marriage of Figaro Overture" 270) ("Gershwin"
  "Summertime" 300) ("Queen" "Bohemian Rhapsody" 355) ("Gershwin" "Rhapsody in Blue" 1100)))
(rf '("Mozart" "The Marriage of Figaro Overture" 270)) → "Summertime"
(rf '("Gershwin" "Summertime" 300)) → "Rhapsody in Blue"
(rf '("Gershwin" "Rhapsody in Blue" 1100)) → "Bohemian Rhapsody"
```

Haskell

```
recommender pl = _____
  let avgDuration artist = _____
      option1 = _____
      option2 = _____
  in if not (null option1) then (_____ option1 )
     else if not (null option2) then (_____ option2 )
     else _____
```

Пример:

```
rf = recommender [("Mozart","The Marriage of Figaro Overture",270), ("Gershwin","Summertime",
  300), ("Queen","Bohemian Rhapsody",355), ("Gershwin","Rhapsody in Blue",1100)]
rf ("Mozart","The Marriage of Figaro Overture",270) → "Summertime"
rf ("Gershwin", "Summertime", 300) → "Rhapsody in Blue"
rf ("Gershwin", "Rhapsody in Blue", 1100) → "Bohemian Rhapsody"
```

Задача 3. Решете задачата на езика *Haskell*.

А) Двоично дърво ще представяме чрез следния тип:

```
data Tree a = EmptyTree | Node {
    value :: a,
    left  :: Tree a,
    right :: Tree a
} deriving (Show,Read)
```

Довършете функцията `treeWords`:

- 1. Опишете типа на функцията така, че тя да получава дърво от символи и да връща списък от символни низове.
- 2. Функцията да връща списък от всички думи, които могат да се образуват по път от корена до някое от листата на дървото. Ако дървото е празно, да се връща празният списък.

```
treeWords :: _____

treeWords _____ = []

treeWords (Node v EmptyTree EmptyTree) = [[_____]]

treeWords (Node v l r) = map (v:) (_____ ++ _____)

where wl = _____

      wr = _____
```

Б) Довършете кода на функцията `quickSort` така, че тя да сортира списък по метода на бързото сортиране (*quicksort*). Опишете типа на функцията по такъв начин, че тя да може да работи със списък от произволен тип елементи, стига те да са от класа *Ord*.

```
quickSort :: _____

quickSort [] = _____

quickSort (x:xs) = _____ ++ _____ ++ _____

where
    lesser = filter _____
    greater = filter _____
```

Задача 3. Задачата да се реши на езика *Haskell*. Отговорите си попълнете в посочените за тях полета. Оценява се само попълненото в полетата. Всичко друго писано по листа не носи точки.

Под всеки от дадените по-долу изрази посочете каква ще бъде неговата оценка. За някои от изразите са дадени възможности, измежду които трябва да изберете. Други трябва да съобразите и да попълните сами. За изразите, които трябва да попълните, точки се дават само за напълно коректни отговори. Ако посоченият от вас отговор не съответства точно на това, което би извел *Haskell* интерпретаторът, той се оценява с нула точки.

А) (1 точка) Оградете оценката на израза:

`foldr1 (&&) [False, False ..]`

- а) True
- б) False
- в) Изразът ще предизвика безкрайно изпълнение.

Б) (1 точка) Оградете оценката на израза:

`foldr1 (&&) [True, True ..]`

- а) True
- б) False
- в) Изразът ще предизвика безкрайно изпълнение.

В) (1 точка) Попълнете оценката на израза:

`filter (`elem` [10..20]) [1,5,10,100,20,15]`

Отговор: _____

Г) (1 точка) Оградете оценката на израза:

`negate $ max 10 20`

- а) 10
- б) 20
- в) -10
- г) -20
- д) Операторът \$
- е) Операторът \$ не може да се използва инфиксно.

Д) (1 точка) Попълнете оценката на израза:

`print (take 4 [1,3..])`

Отговор: _____

Е) (1 точка) Попълнете оценката на израза:

`(:[]) []`

Отговор: _____

Ж) (1 точка) Попълнете оценката на израза:

`map ($ 0) (map (+) [1..5])`

Отговор: _____

З) (1 точка) Нека имаме следната функция:

`f l = [ x+y | x <- l, y <- l]`

Попълнете каква ще бъде оценката на израза:

`f [1,10]`

Отговор: _____

И) (1 точка) Нека имаме следната функция:

`g ([]:_) = []`

`g x = (map head x) : g (map tail x)`

Попълнете каква ще бъде оценката на израза:

`g [[1,2,3],[4,5,6]]`

Отговор: _____

Й) (1 точка) В полето по-долу попълнете какъв е типът на функцията `g` от предишния въпрос. Опишете го така, че функцията да може да работи само и единствено за списъци от списъци, съдържащи елементи от класа `Num`.

Отговор: `g ::` _____

Задача 4. Задачата да се реши на един от езиците Scheme или Haskell.

Група приятели отиват на екскурзия и се редуват да плащат общите сметки. Когато се прибират, установяват, че всеки дължи на някого дребна сума пари. Всяко задължение се представя с кортеж от три елемента: длъжник (низ), сума (цяло положително число) и получател (низ). За Scheme под “кортеж” се има предвид просто списък. “Верига” наричаме последователност от **различни** хора, в която всеки дължи на предишния една и съща сума. “Кръг” наричаме затворена верига, т.е. верига, в която първият дължи на последния същата сума. Приятелите се сещат, че ако в задълженията им има верига, то може последният човек в нея да се издължи директно на първия, а ако има кръг, то всички задължения в него се погасяват. Да се попълнят празните полета по-долу така, че:

- а) функцията `maxByLength` да намира най-дълъг списък в списъка от списъци `ls`;
- б) рекурсивната функция `maxChain` да намира най-дълга верига, завършваща със списъка `chain`, сума `amount` и краен получател (т.е. първи елемент) `final` при даден списък от задължения `dues`;
- в) функцията `maxCircle` да намира най-дълъг кръг при даден списък от задължения `dues`.

За всички функции, ако има няколко различни отговора с еднаква дължина, не е от значение кой от тях ще бъде върнат като резултат, а ако няма нито един отговор, да се връща празният списък.

Упътване: могат да се използват наготово всички функции в R5RS за Scheme и в Prelude за Haskell.

Haskell

```
maxByLength ls = foldr _____ [] ls
maxChain chain@(last:rest) amount final dues
| _____ = chain
| otherwise   = maxByLength [ maxChain _____ amount final dues |
                              _____ <- dues, amount == _____,
                              last == _____, _____ ]
maxCircle dues = maxByLength [ maxChain _____ dues |
                              _____ <- dues ]
```

Пример:

```
dues = [("Georgi",10,"Boyan"),("Boyan",15,"Sia"),("Sia",15,"Maria"),("Maria",10,"Georgi"),
        ("Maria",10,"Petar"),("Petar",10,"Georgi"),("Boyan",10,"Maria")]
maxChain ["Boyan"] 15 "Maria" dues → ["Maria","Sia","Boyan"]
maxCircle dues → ["Boyan","Georgi","Petar","Maria"]
```

Scheme

```
(define (maxByLength ls)
  (foldr _____ '() ls))
(define (maxChain chain amount final dues)
  (let ((last (car chain)) (rest (cdr chain)))
    (if _____ chain
        (maxByLength (map (lambda (t) (maxChain _____ amount final dues))
                          (filter (lambda (t) (and (= amount _____)
                                                    (eqv? last _____)
                                                    _____)) dues))))))
(define (maxCircle dues)
  (maxByLength (map (lambda (t)
                    (maxChain _____ dues) dues))))
```

Пример:

```
(define dues '(("Georgi" 10 "Boyan") ("Boyan" 15 "Sia") ("Sia" 15 "Maria") ("Maria" 10 "Georgi")
               ("Maria" 10 "Petar") ("Petar" 10 "Georgi") ("Boyan" 10 "Maria")))
(maxChain '("Boyan") 15 "Maria" dues) → ("Maria" "Sia" "Boyan")
(maxCircle dues) → ("Boyan" "Georgi" "Petar" "Maria")
```

Задача 4. Крайно кореново дърво, всеки връх на което съдържа цяло число и може да има произволен брой деца, се представя в Scheme като наредена двойка, състояща се от стойността на корена и списък от директни поддървета, а в Haskell — със следната рекурсивна структура:

```
data Tree = T { root :: Int, subtrees :: [Tree] } deriving Show
```

Листо наричаме дърво с единствен връх, **клонка** наричаме дърво, чиито директни поддървета са листа, а **пръчка** наричаме дърво, в което всеки връх има най-много едно дете. Казваме, че едно дърво се **подрязва**, ако от него се премахнат всички клонки, с изключение на корена, и че се **окастрия**, ако всички пръчки в него, които са с повече от два върха и не са част от други пръчки, се скъсят откъм листата до дължина точно два върха. Да се попълнят празните полета по-долу, така че:

- функциите leaf, twig и stick да проверяват дали дърво е съответно листо, клонка или пръчка;
- функциите trim и prune да връщат съответно подрязано или окастриено копие на дървото, подадено им като параметър.

Упътване: могат да се използват наготово all, any, car, cdr, filter, foldl, foldr, head, null, null?, tail, length, map, както и всички функции в R5RS за Scheme и в Prelude за Haskell.

Haskell

```
leaf _____  
twig _____  
stick _____  
  
trim (T x ts) = T x _____  
prune t@(T x []) = _____  
prune t@(T x ts) = T x (if stick t  
                        then _____  
                        else _____)
```

Пример:

```
twig (T 1 [T 2 [], T 3 []]) → True           stick (T 1 [T 2 [T 3 [T 4 []]]) → True  
tree = T 1 [T 2 [T 3 []], T 4 [T 5 [T 6 []]], T 7 [T 8 [], T 9 [T 10 [T 11 []]]]  
trim tree → T 1 [T 4 [], T 7 [T 9 []]]  
prune tree → T 1 [T 2 [T 3 []], T 4 [T 5 []], T 7 [T 8 [], T 9 [T 10 []]]
```

Scheme

```
(define (leaf t) _____)  
(define (twig t) _____)  
(define (stick t) _____)  
(define (trim t)  
  (cons (car t) _____))  
(define (prune t)  
  (if (leaf t) _____  
      (cons (car t) (if (stick t) _____))))
```

Пример:

```
(twig '(1 (2) (3))) → #t           (stick '(1 (2 (3 (4))))) → #t  
(define tree '(1 (2 (3)) (4 (5 (6))) (7 (8) (9 (10 (11)))))  
(trim tree) → (1 (4) (7 (9)))  
(prune tree) → (1 (2 (3)) (4 (5)) (7 (8) (9 (10))))
```

Задача 4. Задачата да се реши на един от езиците Scheme или Haskell.

Обществото на информатиците провежда избори за съвет на старейшините. „Кандидатура“ е наредена двойка от низове: името на кандидата и неговата специалност, а „кандидатската листа“ е списък от кандидатури. Може да се гласува за произволен брой кандидати. Попълнените „бюлетини“ представляват предикати, които приемат кандидатура и връщат истина или лъжа, в зависимост дали се гласува за съответната кандидатура или не. Да се попълнят празните полета по-долу така, че:

- а) votes да връща броя на гласовете от бюлетините ballots за кандидата с име cand;
- б) election да връща списък от наредени двойки от името на кандидата и броя от гласовете за него от бюлетините ballots в реда, в който кандидатите се срещат в листата cl;
- в) sortResults да сортира резултатите, върнати от функцията election, в низходящ ред по броя на гласовете, като кандидатите с равен брой гласове запазват реда си от листата;
- г) selectCouncil да връща списък от имена от избраните **не повече** от max кандидати, които имат повече от половината гласове, в реда, върнат от sortResults.

Упътване: могат да се използват наготово всички функции в R5RS за Scheme и в Prelude за Haskell.

Haskell

```
votes cand ballots = _____
election ballots cl = _____ (\_____ -> (_____, _____)) cl
sortResults [] = []
sortResults _____ = more ++ equal ++ less
  where more = _____
        equal = _____
        less = _____
selectCouncil max ballots cl =
  _____ (map _____ (filter _____ results))
  where results = sortResults (election ballots cl)
```

Пример:

```
cl = [("Kernighan","C"),("Ritchie","C"),("Stroustrup","C++"),("Steele","Scheme"),
      ("Sussman","Scheme"),("Church","Lambda"),("Curry","Lambda")]
b1 (name, specialty) = specialty == "Lambda" || last name == 'e'
b2 (name, specialty) = name == "Church" || head specialty == 'C'
b3 (name, specialty) = length name > 6 && specialty /= "C++"
selectCouncil 2 [b1, b2, b3] cl = ["Ritchie","Kernighan"]
```

Scheme

```
(define (votes cand ballots) _____)
(define (election ballots cl) (_____ (lambda (c) (cons _____ _____)) cl))
(define (sortResults res) (if (null? res) '()
  (let ((more _____)
        (equal _____)
        (less _____))
    (append more equal less))))
(define (selectCouncil max ballots cl)
  (define (select max results) (if (or (null? results) _____) '()
    (cons _____ (select (- max 1) (cdr results)))))
  (select max (sortResults (election ballots cl))))
```

Пример:

```
(define cl '("Kernighan"."C") ("Ritchie"."C") ("Stroustrup"."C++") ("Steele"."Scheme")
  ("Sussman"."Scheme") ("Church"."Lambda") ("Curry"."Lambda"))
(define (b1 c) (or (equal? (cdr c) "Lambda") (eqv? (car (reverse (string->list (car c)))) #\e)))
(define (b2 c) (or (equal? (car c) "Church") (eqv? (string-ref (cdr c) 0) #\C)))
(define (b3 c) (and (> (string-length (car c)) 6) (not (equal? (cdr c) "C++"))))
(selectCouncil 2 (list b1 b2 b3) cl) -> ("Ritchie" "Kernighan")
```

Задача 4. Моделът „ChatFOS 5“ може да генерира прости функционални изрази, съдържащи една променлива *x* и извиквания на двуместни числови функции от предварително зададена „библиотека“, които моделът познава и може да използва. Библиотеката се представя със списък от наредени двойки: символ на функцията и самата функция. Изразите на Haskell са представени с дървета от следния алгебричен тип: `data Expr = N Integer | X | F Char Expr Expr`, където конструкторът *N* представя числова константа, *X* представя променливата на израза, а *F* представя функция със зададен символ и два израза за аргументи. Изразите на Scheme използват синтаксиса за *S*-изрази.

- Да се попълнят празните полета по-долу на един от езиците Haskell или Scheme така, че:
- а) функцията `correct` да проверява дали даденият израз е коректно построен само чрез функциите от библиотеката `lib`, цели числа и символа *x*;
 - б) функцията `calc` да връща едноместната функция, пресмятана от даден израз при библиотека от функции `lib`, като допуска, че изразът е коректно построен;
 - в) функцията `check` да проверява дали генерираният израз `expr` правилно пресмята функцията `f`, като проверява дали те връщат еднакви стойности над всички цели числа в интервала $[a;b]$;
 - г) функцията `score` намира точността на модела (число в интервала $[0; 1]$) като по дадени списъци `exprs` от генерирани изрази и `fs` от функции, които те би трябвало да пресмятат, чрез `check` намира отношението на броя на правилно пресметнатите функции към броя на всички функции.

Упътване: могат да се използват наготово всички функции в *R5RS* за Scheme и в *Prelude* за Haskell.

Haskell

```
correct lib (F f l r) = _____ && _____ && _____
correct lib _       = _____
calc lib (N n)      = _____
calc lib X          = _____
calc lib (F c l r)  = _____

check a b lib f expr = _____ (\x -> _____ && _____) [a..b]
score a b lib fs exprs =  fromIntegral (length (filter _____))
                        / fromIntegral (length _____)
```

Пример:

```
exprs = [F '-' (F '*' X X) (N 1), F '+' (F '*' X (N 2)) (N 3),
         F '*' (F '*' X X) (F '+' X (N 2)), F '+' (F '^' X (N 3)) (N 1)]
lib = [(('+', (+)), ('-', (-)), ('*', (*)))] -- calc lib (head exprs) 3 → 8
score (-10) 10 lib [\x -> x^2-1, \x -> x*3+2, \x -> x^3+2*x^2, \x -> x^3+1] exprs → 0.5
```

Scheme

```
(define (correct lib expr)
  (if (pair? expr) (and _____)
      (or _____)))
(define (calc lib expr) (_____
  (cond ((number? expr) _____)
        ((eqv? expr 'x) _____)
        (else _____))))
(define (check a b lib f expr) (if (> a b) _____
  (and _____ (check (+ a 1) b lib f expr))))
(define (score a b lib fs exprs) (/ (length (filter _____))
  (length _____)))
```

Пример:

```
(define lib (list (cons '+ +) (cons '- -) (cons '* *))) ; ((calc lib '(- (* x x) 1)) 3) → 8
(define fs (list (lambda (x) (- (expt x 2) 1)) (lambda (x) (+ 2 (* x 3)))
  (lambda (x) (+ (expt x 3) (* 2 x x))) (lambda (x) (+ 1 (expt x 3)))))
(score -10 10 lib fs '((- (* x x) 1) (+ (* x 2) 3) (* (* x x) (+ x 2)) (+ (^ x 3) 1))) → 0.5
```

Задача 4. Амбициозната стартап компания „Crypto Quantum AI“ провела масова рекламна кампания в социалната мрежа Y, за да привлече максимален брой потребители за новия си революционен продукт. Мрежата Y предоставя функция `clicks`, която по подаден идентификатор на реклама (низ) връща списък от идентификатори на потребители (цели числа), които са посетили рекламната връзка. За следващата кампания в социалната мрежа Lambda, компанията иска да намали броя на рекламите, за които плаща. Да се реализира функция `redundant`, която по подадена функция `clicks` и множество реклами `ads` връща множество от реклами, за всяка от които е вярно, че ако се премахне от кампанията, множеството на привлечените потребители ще остане без промяна. За целта да се реализират следните помощни функции:

- а) `union`, която намира обединението на две множества от цели числа;
- б) `subset`, която реализира релацията „подмножество“ за множества от цели числа;
- в) `collect`, която по подадена функция `clicks` и списък от реклами връща множество от всички потребители, които са посетили връзката на поне една реклама.

Забележка: В тази задача множествата се представят като списъци без повторения.
Упътване: Позволено е използването на функциите `all`, `elem`, `eqv?`, `filter`, `foldr`, `map`, `memv`, `not`, както и на всички функции от R⁵RS и Prelude.

Haskell

```

union  xs ys = _____
subset xs ys = _____
collect clicks ads = foldr _____ ads
redundant clicks ads = filter (\ad -> subset _____
                                (collect clicks _____)) ads
  
```

Пример:

```

clicks "a" = [1,2,3]; clicks "b" = [1,3,5]; clicks "c" = [2,3,5,7]
union [1,3,4] [4,1,7] → [3,4,1,7]
subset [1,2] [3,2,1]  → True
collect clicks ["a", "b", "c"] → [1,2,3,5,7]
redundant clicks ["a", "b", "c"] → ["a", "b"]
  
```

Scheme

```

(define (union  xs ys) _____)
(define (subset xs ys) _____)
(define (collect clicks ads)
  (foldr _____ ads))
(define (redundant clicks ads)
  (filter (lambda (ad)
            (subset _____ (collect clicks
                                _____)))) ads)
  
```

Пример:

```

(define (clicks ad) (case ad (('a) '(1 2 3)) (('b) '(1 3 5)) (('c) '(2 3 5 7))))
(union '(1 3 4) '(4 1 7)) → (3 4 1 7)
(subset '(1 2) '(3 2 1))  → #t
(collect clicks '("a" "b" "c")) → (1 2 3 5 7)
(redundant clicks '("a" "b" "c")) → ("a" "b")
  
```

Задача 4. Задачата да се реши на един от езиците *Haskell* или *Scheme*.

Нека са дадени фиксирано крайно множество от цели числа *dom* и крайно множество *fs* от едноаргументни целочислени функции с дефиниционно множество *dom*.

Казваме, че функцията *f* е *затворена над dom*, ако дава само стойности в *dom* при аргументи от *dom*.

Казваме, че функциите *fs* *образуват моноид*, ако:

- (1) всички функции от *fs* са затворени в *dom*;
- (2) *fs* съдържа функция, която е идентитет над дефиниционното множество *dom*;
- (3) за всеки две функции от *fs* е вярно, че композицията им също е във *fs*.

Да се попълнят празните места по-долу, така че:

- а) *eqFn* да проверява дали две функции *f* и *g* съвпадат върху множеството *dom*;
- б) *memFn* да проверява дали във *fs* може да се намери функция равна на *f* в смисъла на *eqFn*;
- в) *checkRan* да проверява дали *f* е затворена над *dom*;
- г) *isMonoid* да проверява дали функциите в списъка *fs*, ограничени върху дефиниционното множество *dom*, образуват моноид съгласно условията (1), (2) и (3).

Забележка: Считаме, че *dom* е дефинирано глобално.

Упътване: Позволено е използването на функциите *all*, *any*, *elem*, *filter*, *foldr*, *map*, *memv*, както и на всички функции от *R⁵RS* и *Prelude*.

Haskell

```
eqFn f g   = _____ dom
memFn f fs = _____ fs
checkRan f = _____ (_____ f dom)
isMonoid fs = _____ checkRan fs &&
                memFn _____ fs &&
                _____ (\f -> _____ (\g -> memFn _____ fs) fs) fs
```

Пример:

```
dom = [0, 1, 2]; monoid = [(`mod` 3) . (+1), (`mod` 3) . (+2), id]
isMonoid monoid → True
isMonoid (tail monoid) → False
isMonoid ((*2) : monoid) → False
```

Scheme

```
(define (all p l) (not (memv #f (map p l))))
(define (any p l) (not (null? (filter p l))))
(define (eqFn f g) (_____ dom))
(define (memFn f fs) (_____ fs))
(define (checkRan f) (_____ (_____ f dom)))
(define (isMonoid fs)
  (and (_____ checkRan fs)
        (memFn _____ fs)
        (_____ (lambda (f)
                    (_____ (lambda (g)
                              (memFn _____ fs)) fs)) fs)))
```

Пример:

```
(define dom '(0 1 2))
(define monoid (list (lambda (x) (remainder (+ x 1) 3)) (lambda (x) (remainder (+ x 2) 3))
                    (lambda (x) x)))
(isMonoid monoid) → #t
(isMonoid (cdr monoid)) → #f
(isMonoid (cons (lambda (x) (* x 2)) monoid)) → #f
```